

08. profile YAML 読込と検証

solar_ws0129-main

2026-07-29

対象

項目	内容
ファイル	mpc_solarcar/solar_profile.py
種別	Python
区分	config
役割	profile.yaml をロードし、セクション取得、相対パス解決、CSV 最低品質検査を行う。
起動文脈	launch、offline simulation、identification の共通設定入口。

このファイルを読む理由

profile.yaml をロードし、セクション取得、相対パス解決、CSV 最低品質検査を行う。

- load_profile が YAML 全体を返す。
- get_path が profile 基準で実ファイルパスへ変換する。
- require_csv_data_rows が空テンプレと実データを区別する。

呼び出し関係

どこから呼ばれるか

- live_launch.py
- solarcar_sim.launch.py
- solar_state_node.py
- 多数の scripts

次に読むと理解が進むファイル

- mpc_solarcar/path_utils.py

同一リポジトリ内の主要依存

- 同一リポジトリ内の明示 import は少ない。

ソース冒頭の把握

モジュール docstring または先頭意図の要約:

モジュール docstring は明示されていない。

代表コード断片

```
ROOT = Path(__file__).resolve().parents[1]

def require_csv_data_rows(
    path: str | Path,
    *,
    label: str,
    required_columns: tuple[str, ...] = (),
) -> Path:
    resolved = Path(path).expanduser().resolve()
    if not resolved.is_file():
        raise FileNotFoundError(f"{label} CSV was not found: {resolved}")
    with resolved.open("r", encoding="utf-8-sig", newline="") as stream:
        reader = csv.DictReader(stream)
        header = tuple(reader.fieldnames or ())
        missing = [column for column in required_columns if column not in header]
        if missing:
            raise ValueError(f"{label} CSV is missing columns {missing}: {resolved}")
        if next(reader, None) is None:
            raise ValueError(
                f"{label} CSV has a header but no data rows: {resolved}. "
                "Fill the template or select an identified vehicle profile before launch."
```

主要構造の読み取り

主要関数は `require_csv_data_rows`, `resolve_relative_path`, `load_profile`, `merged_dict`, `get_section`, `get_path`。

主要クラス・関数

行	種別	名前	補足
14	function	<code>require_csv_data_rows</code>	args: path
37	function	<code>_resolve_profile_path</code>	args: path_like
51	function	<code>resolve_relative_path</code>	args: base_dir, path_like
68	function	<code>load_profile</code>	args: path_like
77	function	<code>merged_dict</code>	
78	function	<code>_merge</code>	args: dst, src
93	function	<code>get_section</code>	args: cfg, key, default
108	function	<code>get_path</code>	args: cfg, profile_path, key, default

ファイルを上から読んだときの定義順

行	内容
11	ROOT に <code>Path(__file__).resolve().parents[1]</code> の結果を代入する。

14	関数 <code>require_csv_data_rows</code> を定義する。
37	関数 <code>_resolve_profile_path</code> を定義する。
51	関数 <code>resolve_relative_path</code> を定義する。
68	関数 <code>load_profile</code> を定義する。
77	関数 <code>merged_dict</code> を定義する。
93	関数 <code>get_section</code> を定義する。
108	関数 <code>get_path</code> を定義する。

import 群の詳細

行	import 文	このファイルで読む理由
1	<code>from __future_ _import annotations</code>	型ヒントの遅延評価を有効にし、自己参照型や forward reference を安全に扱うため。このファイル内での使用位置は少ないか、間接利用である。
3	<code>import copy</code>	設定辞書や <code>payload</code> を安全に複製するため。このファイル内での主な使用位置は L81, L83, L99, L102, L104, L105。
4	<code>import csv</code>	CSV の逐次読込・逐次書込を行うため。このファイル内での主な使用位置は L24。
5	<code>from pathlib import Path</code>	ファイルやディレクトリを安全に扱うため。このファイル内での主な使用位置は L11, L15, L19, L20, L37, L38, L42, L51, ...。
6	<code>from typing import Any</code>	<code>typing</code> から <code>Any</code> を読み込み、このファイルの処理を組み立てるため。このファイル内での主な使用位置は L86, L93, L94。
8	<code>import yaml</code>	<code>profile</code> 、 <code>stop</code> 、 <code>schedule</code> 、 <code>summary</code> YAML を読み書きするため。このファイル内での主な使用位置は L71。

関数・クラスを上から順に解説

L14 関数 `require_csv_data_rows`

- 定義: `require_csv_data_rows(path, label, required_columns)`
 - docstring: 明示されていない。
 - このブロックが直接呼ぶ主な関数・メソッド: `DictReader`, `FileNotFoundError`, `Path`, `ValueError`, `expanduser`, `is_file`, `next`, `open`, `resolve`, `tuple`
 - 戻り値の要点: `resolved`
1. `resolved` に `Path(path).expanduser().resolve()` の結果を代入する。
 2. 条件 `not resolved.is_file()` を判定し、真なら内部処理を行う。
 3. `with` 文で `resolved.open('r', encoding='utf-8-sig', newline='')` を管理しながら処理する。
 4. `resolved` を返す。

L37 関数 `_resolve_profile_path`

- 定義: `_resolve_profile_path(path_like)`
- docstring: 明示されていない。
- このブロックが直接呼ぶ主な関数・メソッド: `Path`, `cwd`, `exists`, `expanduser`, `is_absolute`, `resolve`, `str`, `strip`
- 戻り値の要点: `candidates[-1] / raw.resolve() / candidate`

1. `raw` に `Path(str(path_like or "").strip()).expanduser()` の結果を代入する。
2. 条件 `raw.is_absolute()` を判定し、真なら内部処理を行う。
3. `candidates` に `[(Path.cwd() / raw).resolve(), (ROOT / raw).resolve()]` の結果を代入する。
4. `candidates` を順に走査し、各要素を `candidate` に入れて処理する。
5. `candidates[-1]` を返す。

L51 関数 `resolve_relative_path`

- 定義: `resolve_relative_path(base_dir, path_like)`
- docstring: 明示されていない。
- このブロックが直接呼ぶ主な関数・メソッド: `Path`, `exists`, `expanduser`, `is_absolute`, `resolve`, `str`, `strip`
- 戻り値の要点: `str(candidate) / "" / str(path.resolve()) / str(candidate)`

1. `raw` に `str(path_like or "").strip()` の結果を代入する。
2. 条件 `not raw` を判定し、真なら内部処理を行う。
3. `path` に `Path(raw).expanduser()` の結果を代入する。
4. 条件 `path.is_absolute()` を判定し、真なら内部処理を行う。
5. `base` に `Path(base_dir).resolve()` の結果を代入する。
6. `candidate` に `(base / path).resolve()` の結果を代入する。
7. 条件 `candidate.exists()` を判定し、真なら内部処理を行う。
8. `repo_candidate` に `(ROOT / path).resolve()` の結果を代入する。
9. 条件 `repo_candidate.exists()` を判定し、真なら内部処理を行う。
10. `str(candidate)` を返す。

L68 関数 `load_profile`

- 定義: `load_profile(path_like)`
- docstring: 明示されていない。
- このブロックが直接呼ぶ主な関数・メソッド: `ValueError`, `_resolve_profile_path`, `isinstance`, `open`, `safe_load`
- 戻り値の要点: `(profile_path, cfg)`

1. `profile_path` に `_resolve_profile_path(path_like)` の結果を代入する。

2. with 文で `profile_path.open('r', encoding='utf-8')` を管理しながら処理する。
3. 条件 `not isinstance(cfg, dict)` を判定し、真なら内部処理を行う。
4. `(profile_path, cfg)` を返す。

L77 関数 `merged_dict`

- 定義: `merged_dict(*payloads)`
 - docstring: 明示されていない。
 - このブロックが直接呼ぶ主な関数・メソッド: `_merge, deepcopy, get, isinstance, items`
 - 戻り値の要点: `out / dst`
1. 関数 `_merge` を定義する。
 2. `out` に `{}` を代入する。
 3. `payloads` を順に走査し、各要素を `payload` に入れて処理する。
 4. `out` を返す。

L93 関数 `get_section`

- 定義: `get_section(cfg, key, default)`
 - docstring: 明示されていない。
 - このブロックが直接呼ぶ主な関数・メソッド: `deepcopy, get, isinstance, split, str`
 - 戻り値の要点: `copy.deepcopy(current) / copy.deepcopy(default) if default is not None else {} / copy.deepcopy(default) if default is not None else {} / copy.deepcopy(default) if default is not None else {}`
1. `current` に `cfg` を代入する。
 2. `str(key or "").split('.')` を順に走査し、各要素を `part` に入れて処理する。
 3. 条件 `current is None` を判定し、真なら内部処理を行う。
 4. `copy.deepcopy(current)` を返す。

L108 関数 `get_path`

- 定義: `get_path(cfg, profile_path, key, default)`
 - docstring: 明示されていない。
 - このブロックが直接呼ぶ主な関数・メソッド: `Path, get, get_section, isinstance, resolve, resolve_relative_path, str, strip`
 - 戻り値の要点: `resolve_relative_path(profile_dir, raw)`
1. `profile_dir` に `Path(profile_path).resolve().parent` の結果を代入する。
 2. `raw` に `”` の結果を代入する。
 3. `paths_cfg` に `cfg.get('paths', {})` if `isinstance(cfg, dict)` else `{}` の結果を代入する。
 4. 条件 `isinstance(paths_cfg, dict)` を判定し、真なら内部処理を行う。

5. 条件 `not raw` を判定し、真なら内部処理を行う。
6. 条件 `not raw` を判定し、真なら内部処理を行う。
7. `resolve_relative_path(profile_dir, raw)` を返す。

処理の流れ

1. ソース中の主要関数を通じて処理を進める。

このファイルの位置づけ

この資料群では、`mpc_solarcar/solar_profile.py` を **config** の中核として扱う。したがって、ここで扱う処理は単独で完結せず、前段の `profile / launch / telemetry` と後段の `planner / logger / report` へ繋がる。